

函数式程序设计

主要内容

- 命令式vs声明式程序设计范式
- 什么是函数式程序设计
- 函数式程序设计基本手段
- 逻辑式程序设计简介

命令式程序设计范式 (imperative programming)

- 命令式程序设计范式是指：
 - 针对一个目标，需要给出达到目标的操作步骤和状态变化，即要对“如何做”进行详细描述。
 - 它们与冯诺依曼体系结构一致，是使用较广泛的程序设计范式，适合于解决大部分的实际应用问题。
- 命令式程序设计范式的典型代表：
 - 过程式程序设计
 - 面向对象程序设计

声明式程序设计范式 (declarative programming)

- 声明式程序设计范式是指：
 - 只需要给出目标的定义，不需要对如何达到目标（包括操作步骤和状态变化）进行描述，即只需要对“做什么”进行描述。
 - 有良好的数学理论支持，易于保证程序的正确性，并且，设计出的程序比较精炼和具有潜在的并行性。
- 声明式程序设计范式的典型代表：
 - 函数式程序设计
 - 逻辑式程序设计

例：命令式 VS 声明式

■ 计算：

$a * b + c / d$

• 命令式编程：

$t1 = a * b;$

$t2 = c / d;$

$r = t1 + t2;$

• 函数式编程：

... **add**(**multiply**(a,b),**divide**(c,d)) ...

■ 计算一系列数的和：

$$a_1 + a_2 + a_3 + \dots + a_N$$

• 命令式编程：

```
int a[N]={a1, a2, a3,..., aN},sum=0;  
for (int i=0; i<N; i++) sum += a[i];  
cout << sum;
```

• 函数式编程：

```
int sum(int x[], int n)  
{ if (n==1) return x[0];  
  else return x[0]+sum(x+1,n-1);  
}  
int a[N]={a1, a2, a3,..., aN};  
cout << sum(a,N);
```

什么是函数式程序设计

- 函数式程序设计（functional programming）是指把程序组织成一组数学函数，计算过程体现为基于一系列函数应用（把函数作用于数据）的表达式求值。
- 函数也被作为值（数据）来看待，即函数的参数和返回值也可以是函数。
- 基于的理论是递归函数理论和lambda演算。

函数式程序设计的基本特征

- “纯”函数：
 - 以相同的参数调用一个函数总得到相同的值。（引用透明）
 - 除了产生计算结果，不会改变其他任何东西。（无副作用）
- 没有状态：
 - 计算体现为数据之间的映射，它不改变已有数据，而是产生新的数据。（无赋值操作）
- 函数也是值：
 - 函数的参数和返回值都可以是函数，可由已有函数生成新的函数。（高阶函数）
- 递归是主要的控制结构：
 - 重复操作采用函数的递归调用来实现，而不采用迭代（循环）。

- 表达式的惰性（延迟）求值（Lazy evaluation）：
 - 一个表达式只有需要用到它的值的时候才会去计算它。
- 潜在的并行性：
 - 由于程序没有状态以及函数的引用透明和无副作用等特点，因此一些操作可以并行执行。

- 计算字符串的逆序：

"abcd" --> "dcba"

•命令式编程：

```
void reverse(char str[]) //修改已有数据
{ for (int i=0,j=strlen(str)-1; i<j; i++,j--)
  { char t=str[i];
    str[i] = str[j]; str[j] = t;
  }
}
```

•函数式编程：

```
string reverse(string str) //不修改已有数据，而是产生新数据
{ if (len(str) == 1)
  return str;
else
  return concat(reverse(substr(str,1,len(str)-1)),
                substr(str,0,1));
} //concat：字符串拼接；substr：求子串
```

- 注意：不是写成函数就是函数式编程！

■ 计算一系列数中偶数的个数。

- 命令式编程：

```
int a[N]={a1, a2, a3,..., aN},count=0;  
for (int i=0; i<N; i++) //指定操作步骤  
    if (a[i]%2 == 0) count++;  
cout << count;
```

- 函数式编程：

```
bool f(int x) { return x%2 == 0; }  
vector<int> v={a1, a2, a3,..., aN};  
cout << count_if(v.begin(),v.end(),f);
```

函数式编程语言

- 纯函数式编程语言：
 - Common Lisp, Scheme, Clojure, Racket, Erlang, OCaml, Haskell, F#
- 对函数式编程提供支持的语言：
 - Perl, PHP, C# 3.0, Java 8, Python, C++(11)
- C++是个多范式编程语言：
 - 除了支持过程式和面向对象程序设计范式外，
 - 也支持函数式程序设计范式。
- 在C++中，函数式编程主要通过STL来实现。

函数式程序设计的基本手段

- 递归和尾递归
- 过滤/映射/规约操作
(Filter/Map/Reduce)
- 部分函数应用 (Partial Function Application)
- 柯里化 (Currying)
-

递归

- 实现重复操作，不采用迭代方式（循环），而是采用递归。例如，求第n个fibonacci数：

- 命令式方案（迭代）

```
int fib_1=1,fib_2=1,n=10;  
for (int i=3; i<=n; i++) //求第10个fibonacci数  
{ int temp=fib_1+fib_2;  
  fib_1 = fib_2; fib_2 = temp;  
}  
cout << fib_2 << endl;
```

- 函数式方案1（递归）

```
int fib(int n)  
{ if (n == 1 || n == 2) return 1;  
  else return fib(n-2)+fib(n-1);  
}  
cout << fib(10) << endl; //求第10个fibonacci数
```

尾递归

- 由于函数递归调用效率低、递归调用深度受栈空间的限制，因此，函数式编程常采用尾递归，即递归调用是函数的最后一步操作。

- 求第n个fibonacci数的函数式方案2（尾递归）

```
int fib(int n, int a, int b)
{ if (n == 1) return a;
  else if (n == 2) return b;
  else return fib(n-1,b,a+b);
}
```

```
cout << fib(10,1,1) << endl; //求第10个fibonacci数
```

- 注意：不是函数体中只有一次递归调用就是尾递归！例如，下面的求n!的函数就不是尾递归：

```
int factorial(int n)
{ if (n == 1) return 1;
  else return n*factorial(n-1);
}
```

■ 尾递归调用便于编译程序优化：

- 由于递归调用后不再做其它事，从而不会再使用当前栈空间的内容，因此，递归调用时不必额外分配栈空间，可重用当前的栈空间。
- 可以自动转成迭代。

尾递归自动转迭代

```
int fib(int n, int a, int b)
{ while (true)
  { if (n == 1) return a;
    else if (n == 2) return b;
    else //return fib(n-1,b,a+b);
    { int t1=n-1,t2=b,t3=a+b;
      n = t1; a = t2; b = t3;
    }
  }
}
```

- 一般形式的尾递归：

```
T f(T1 x1, T2 x2, ...)  
{  
  .....  
  ... return f(m1,m2,...);  
  .....  
  ... return f(n1,n2,...);  
  .....  
}
```

- 转换成等价的迭代：

```
T f(T1 x1, T2 x2, ...)  
{  
  while (true)  
  {  
    .....  
    ... { T1 t1=m1; T2 t2=m2; ...  
          x1 = t1; x2 = t2; ... continue;}  
    .....  
    ... { T1 t1=n1; T2 t2=n2; ...  
          x1 = t1; x2 = t2; ... continue;}  
    .....  
  }  
}
```

过滤操作 (Filter)

- 把一个集合中满足某条件的元素选出来，构成一个新的集合。

- 例如，求某整数集合中的所有正整数：

```
vector<int> nums={1,-2,7,0,3}, positives;  
copy_if(nums.begin(),nums.end(),  
         back_inserter(positives),  
         [](int x){return x>0;});  
for_each(positives.begin(),positives.end(),  
         [](int x){ cout << x <<','; });
```

映射操作 (Map)

- 对一个集合中的每个元素分别进行某个操作，结果放到一个新集合中。
- 例如，求某整数集合中各整数的平方：

```
vector<int> nums={1,-2,7,0,3}, squares;  
transform(nums.begin(),nums.end(),  
            back_inserter(squares),  
            [](int x){return x*x;});  
for_each(squares.begin(), squares.end(),  
          [](int x){cout<<x<<',';});
```

 - 便于并行优化

规约操作 (Reduce)

- 对一个集合中的所有元素连续进行某个操作，最后得到一个值。

- 例如，计算某整数集合中所有整数的和

```
vector<int> nums={1,-2,7,0,3};  
int sum = accumulate(nums.begin(), nums.end(),  
                      0, [](int a,int x) {return a+x;});  
cout << sum;
```

- 运用过滤/映射/规约操作来实现：输出学生中所有女生的名字和平均年龄：

```
vector<Student> students,students_2;  
vector<string> names;  
//先筛选出“女生”  
copy_if(students.begin(),students.end(),  
        back_inserter(students_2),  
        [](Student &st) { return st.get_sex()==FEMALE;});  
//生成所有女生的名字  
transform(students_2.begin(),students_2.end(),  
          back_inserter(names),  
          [](Student &st) { return st.get_name(); });  
//对名字进行排序并输出  
sort(names.begin(),names.end());  
for_each(names.begin(),names.end(),  
          [](string& s) { cout << s << endl; });  
//求平均年龄  
cout << accumulate(students_2.begin(),students_2.end(),0,  
                    [](int a,Student &st) {return a+st.get_age();})/students_2.size();
```

部分（偏）函数应用 (Partial Function Application)

- 对于一个多参数的函数，在某些应用场景下，它的一些参数往往取固定的值。
- **部分函数应用**是指：对于一个多参数的函数，只给它的某些参数提供值，从而生成一个新函数，该新函数不包含原函数中已提供值的参数。
- 部分函数应用可缩小一个函数的适用范围，提高函数的针对性。

- 例如，对于下面的print函数：

```
void print(int n,int base); //按base指定的进制输出n
```

- 由于它常常用来按十进制输出，因此，可以基于print生成一个新函数**print10**，它把print的参数base固定为10，只接受一个参数n：

```
#include <functional>
```

```
function<void(int)> print10=  
    [](int n) { return print(n,10); };
```

或者

```
auto print10=[](int n) { return print(n,10); };
```

.....

```
print10(23); //等价于：print(23,10);
```


- 上面的print10也可以通过STL中的算法bind以一种更通用的方式来实现：

Fty2 **bind**(Fty1 fn, T1 t1, T2 t2, ..., TN tn);

- fn为一个带n个参数的函数（或函数对象）；

- t1 ~ tn是给函数fn提供的参数值，其中的ti：

- 可以是绑定的一个固定值；
- 也可以是未绑定的值，未绑定的值用下划线加上一个数字这样的特殊名称来表示（如：_1、_2、...等，在名空间std::placeholders中定义），其中的数字表示未绑定值的参数在新函数参数表中的位置；

- bind返回一个新函数，该函数的参数表由函数fn的所有未绑定值的参数构成。

- 例如，下面是基于bind实现的print10：

```
#include <functional>
```

```
using namespace std;
```

```
using namespace std::placeholders;
```

```
function<void(int)> print10=bind(print,_1,10);
```

或者

```
auto print10=bind(print,_1,10);
```

```
.....
```

```
print10(23); //等价于：print(23,10);
```

- 再例如，下面的f2是把product的第二个参数y绑定为4之后得到的函数，该函数的第一个参数为product的参数x，第二个参数为product的参数z：

```
#include <functional>
using namespace std;
using namespace std::placeholders;

//_1、_2、...的定义所在的名空间
double product(double x, double y, double z)
{ cout << x << "*" << y << "*" << z << endl;
  return x*y*z;
}

.....
function<double(double,double)> f2=bind(product,_1,4,_2);
//bind返回一个带两个参数的函数：x和z
f2(3,5); //把bind返回的函数作用于(3,5)，输出：3*4*5
```

- 再例如，下面的f3是把product的第二个参数y绑定为4之后得到的函数，该函数的第一个参数为product的参数z，第二个参数为product的参数x：

```
#include <functional>
using namespace std;
using namespace std::placeholders;

//_1、_2、...的定义所在的名空间
double product(double x, double y, double z)
{ cout << x << "*" << y << "*" << z << endl;
  return x*y*z;
}
.....
function<double(double,double)> f3=bind(product,_2,4,_1);
//bind返回一个带两个参数的函数：z和x
f3(3,5); //把bind返回的函数作用于(3,5)，输出：5*4*3
```

- 部分函数应用可缩小一个函数的适用范围，提高函数的针对性。例如，可以基于下面的函数add生成两个特殊的函数add1和add10，它们分别实现加一和加十的功能：

```
#include <functional>
using namespace std;
using namespace std::placeholders;
int add(int x,int y)
{ return x+y;
}
auto add1=bind(add,_1,1); //得到一个加1的函数
auto add10=bind(add,_1,10); //得到一个加10的函数
.....
b = add1(a); //b为a的值加1
c = add10(a); //c为a的值加10
```

柯里化 (Currying)

- 以数学家和逻辑学家Haskell Curry的名字来命名的。
- 柯里化是指把一个多参数的函数变换成一系列单参数的函数，它们分别接收原函数的第一个参数、第二个参数、.....。
- 柯里化的意义：
 - 数学上：对单参数函数的研究模型可以用到多参数函数上。
 - 对程序设计：不必把一个多参数的函数所需要的参数同时提供给它，可以逐步提供。

- 例如，对于下面带两个参数的函数add：
`int add(int x,int y) { return x+y; }`
- 可把它柯里化成一个单参数函数add_cd（参数为函数add的第一个参数x），该函数返回另一个单参数函数（参数为函数add的第二个参数y）
`function<int (int)> add_cd(int x) //返回值是个单参数函数`
`//或者, auto add_cd(int x)`
`{ return bind(add,x,_1);`
`//或`
`return [x](int y)->int { return add(x,y); };`
`}`
`.....`
`cout << add_cd(1)(2);`
•等价于：
`cout << add(1,2);`

- 利用函数的柯里化，也可以缩小一个函数的适用范围，提高函数的针对性。
- 例如，利用上面柯里化后的函数add_cd也可以生成两个函数：

```
auto add1=add_cd(1); //得到一个加1的函数
```

```
auto add10=add_cd(10); //得到一个加10的函数
```

- 然后，利用它们来实现加一和加十的功能：

```
b = add1(a); //b为a的值加1
```

```
c = add10(a); //c为a的值加10
```


部分函数应用和柯里化的区别

- 虽然部分函数应用和柯里化都能实现类似的功能，但它们是有区别的：
 - 部分函数应用得到的是一个参数个数较少的函数，对该函数的调用将得到一个具体的值；而柯里化得到的则是一个由一系列单参数的函数所构成的函数链，对柯里化后的函数调用将得到函数链上的下一个函数，对函数链上最后一个函数的调用才会得到具体的值。
 - 部分函数应用可以按任意次序绑定原函数的参数值，而柯里化只能依次绑定原函数的参数值。

逻辑式程序设计简介

- 程序由一组**事实**和一组**推理规则**构成，在事实基础上运用推理规则来实施计算。
- 基于的理论是谓词演算。
- 特征
 - 数据（事实和规则）就是程序。
 - 计算（匹配、搜索、回溯）由实现系统自动完成。
- 语言
 - **Prolog**, ALF, Fril, Gödel, Mercury, Oz, Ciao, Visual Prolog, XSB

例：判断两城市之间是否有通路

//事实

path(a,b)

path(c,d)

.....

//规则

path(X,Y):-path(Y,X)

path(X,Y):-path(X,Z),path(Z,Y)

.....

//计算

?-path(m,n)

输出true或false

例：判断亲属关系

father(a,b)

father(c,d)

brother(a,c)

.....

brother(X,Y):-brother(Y,X)

uncle(X,Y):-brother(X,Z),father(Z,Y)

.....

?-uncle(a,U)

输出U的值